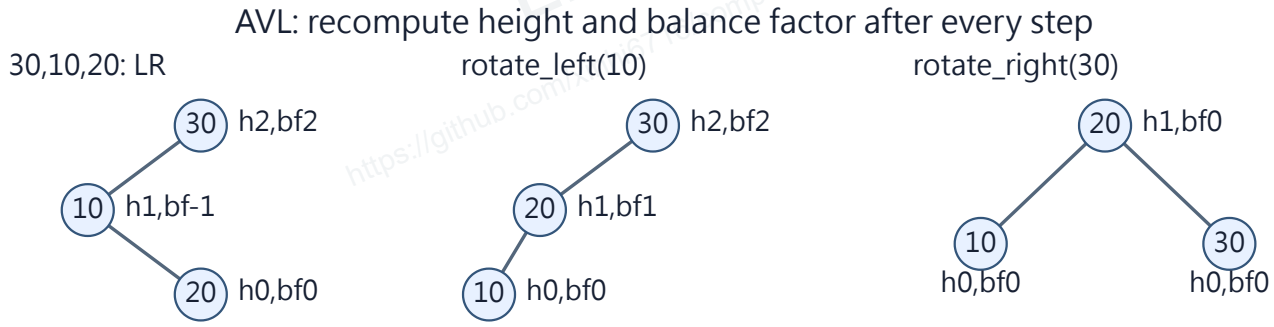
Chapter 10 worked solutions

DS10-E01 • Optimal BST interval DP

Singleton costs are C11=0.1, C22=0.2, C33=0.7. Interval 1..2 has total weight 0.3. Root 1 costs 0.3+0.2=0.5, root 2 costs 0.3+0.1=0.4, so choose 2. Interval 2..3 has weight 0.9. Root 2 costs 1.6 and root 3 costs 1.1, so choose 3. For the whole interval, candidates 1,2,3 cost 2.1,1.8,1.4. The optimum has root 3, left child 2, and its left child 1. If a failed query also pays a dummy comparison, initialize empty e=q. Counting only real-key comparisons removes one comparison from each failed query. Adjust initialization and interpretation consistently instead of mixing the two conventions.

DS10-E02 • AVL insertion versus deletion

After inserting 30, height is 0. Adding 10 gives 30.bf=1 and height 1. Adding 20 gives 10.bf=−1 and 30.bf=2, an LR case. Left-rotate 10 then right-rotate 30. Root 20 has children 10,30 and height 1. For deletion, use root 4, left child 2 with children 1,3, and right leaf 5. Originally root bf=1. Deleting 5 gives root bf=2 while child 2 has bf=0. Use one right rotation, giving root 2, left 1, right 4, and 4's left child 3. The zero child factor is a deletion case and does not require a double rotation. Whether repair continues upward depends on the resulting subtree height change.

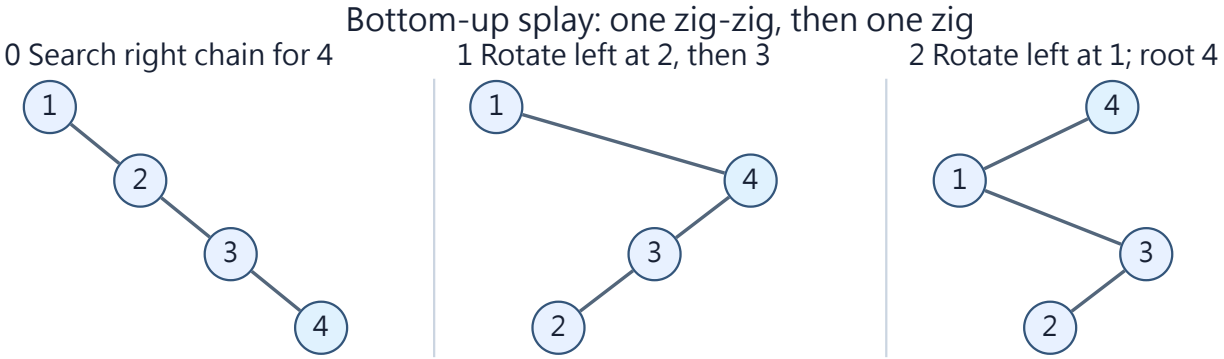


DS10-E03 • Red-black black height and insertion

Color 10 black. Add red 5 on its left. Add red 1 on 5's left. With red parent and black NIL uncle, right-rotate 10 and recolor to black root 5, red children 1,10. Add red 7 as 10's left child. Parent 10 and uncle 1 are red, so color them black and grandparent 5 red, then restore the root to black. Finally 5 is black, 1 and 10 are black, and 7 is red beneath 10. Every root-to-NIL path has three black nodes including root and NIL. This example happens to have only the left-leaning red link 10→7. General red-black validity does not imply that constraint. A black root with a red right child is valid generally but violates LLRB.

DS10-E04 • Splay amortized bounds

With x=4, parent=3, grandparent=2, both steps lean right. Left-rotate 2, then left-rotate 3. The local root is 4 with left child 3 and 3's left child 2, still attached to the right of 1. One final left zig over 1 gives root 4, left child 1, 1's right child 3, and 3's left child 2.



A single search can traverse a Θ(n) chain. The identity is $\sum c_i = \sum \hat{c}_i + \Phi_{\text{initial}} - \Phi_{\text{final}}$, not an individual small c_i guarantee. With unit weights on an arbitrary fixed n-node initial tree, Φ_{initial} can be O(n log n), giving the generic O(m log n+n log n) bound for m accesses. Starting from empty removes the initial term when construction operations are included in the update analysis.

DS10-E05 • Local proof of red-black deletion repair

For this local proof, count black nodes including the subtree root. Let x's actual count be b, becoming b+1 after compensating the deficit. Sibling w has actual count b+1. Since w is black, both nephew subtrees have count b. Coloring the red far nephew black raises its count to b+1. After rotation, p is black and its children are x and the old near-nephew subtree, each of count b, so p's subtree also has b+1. Let c=0 for original red p and c=1 for original black p. Promoted w takes that color and the whole local subtree has black count c+b+1, exactly the predeletion external requirement. If p was red, promoted w is red but its children p and the far nephew are now black, so no red-red violation appears. If p was black, w is black. The deficit is removed locally. This explicitly includes roots in its count and must not be confused with the slides' root-excluding bh definition.

DS10-E06 • Successor transplant, color, and element identity

Node z is 20 and has two non-NIL children. Successor y is node 25, originally red; x=y.right=NIL. Because y.parent is 30 rather than z, transplant(25,NIL) first sets 30.left=NIL and the deficient-position sentinel's parent to 30. Set 25.right=30 and 30.parent=25. Transplant(20,25) gives root=25 and 25.parent=NIL. Attach 25.left=10 with 10.parent=25, copy 20's black color to 25, and release node 20.

The result is black root 25, black left child 10, black right child 30, and red 40 as 30's right child. Removed_color is the red color from 25's old location, so delete_fix does not run. Coloring 25 black at the root replaces the old black root rather than creating a deficit elsewhere. Paths 25B→10B→NIL,25B→30B→NIL, and 25B→30B→40R→NIL each contain three black nodes including root and NIL. Inorder changes from 10,20,25,30,40 to 10,25,30,40.

Node 25 moves without changing identity, preserving its existing handle. Copying its contents into node 20 and deleting the original node 25 gives the same key set but invalidates the original 25 handle and changes what the original 20 handle observes. That violates the specified identity contract. Using z's original black color to invoke repair would also apply the deficit loop where no black deficit exists.

DS10-E07 • Solution: the rotation order matters

(1) Right-rotate 30, then 20. In node order [10,20,30], sizes are [3,5,7], [3,7,3], [7,5,3]. Four leaf ranks are zero. Both endpoint potentials equal $\log_2 105 \approx 6.714$, giving $\Delta\Phi=0$ and amortized cost 2. The final tree is T(10,5,T(20,15,T(30,25,35))).

(2) Left-rotate 10, then right-rotate 30. In order [20,10,30], sizes are [3,5,7], [5,3,7], [7,3,3]. Initial potential is $\log_2 105$ and final potential is $\log_2 63 \approx 5.977$. Thus $\Delta\Phi=\log_2(3/5) \approx -0.737$ and amortized cost is about $1.263 \leq 2\log_2(7/3) \approx 2.445$. The final tree is T(20,T(10,5,15),T(30,25,35)).

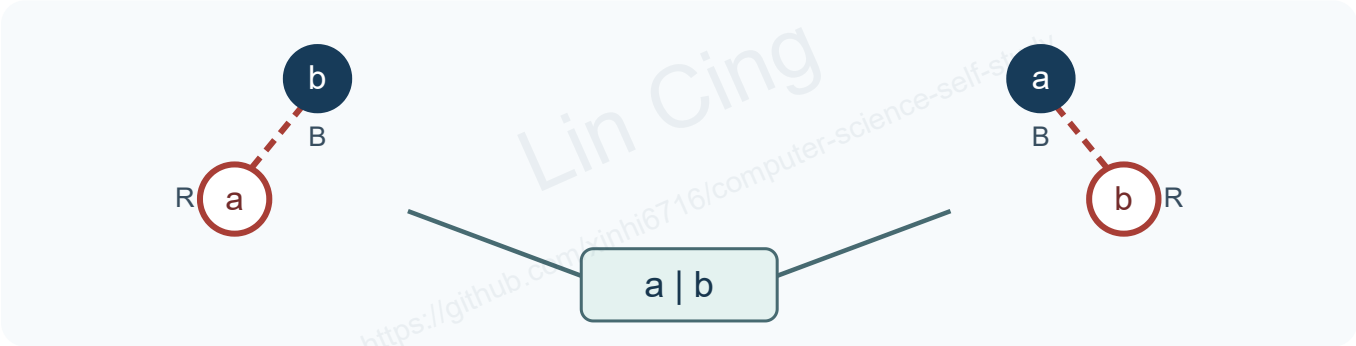
(3) In case (1), old x has {5,10,15} and new g has {25,30,35}. In case (2), new p and new g contain those respective sets. They are disjoint. Reversing the first case's order, right-rotate 20 then 30, instead gives T(10,5,T(30,T(20,15,25),35)). Old x and this new g overlap at node 15, so the standard disjoint-set argument cannot be reused. This shows that the proof does not apply; this one example alone does not establish the alternate rule's sequence complexity.

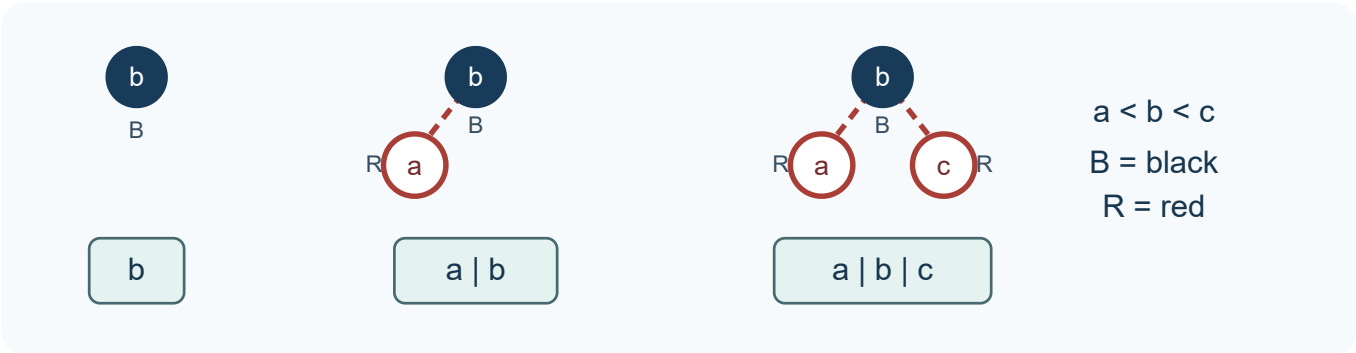
DS10-E08 • AVL height and node count

At least one child has height h−1 and the other at least h−2; minimum children attain N(h)=1+N(h−1)+N(h−2), with bases 0 and 1. N(9)=143 and N(10)=232, so the maximum height is at most 9. Join two height-8 subtrees of sizes 88 and 111 to a root to attain height 9 with 200 nodes. A size-111 subtree exists: start from a minimum 88-node AVL tree of height 8 and insert 23 distinct keys. Its height cannot reach 9 with fewer than 143 nodes. Under leaf height 1 the answer is 10.

DS10-E09 • Red-black and multiway trees

The 3-node is either black b with red left child a, or black a with red right child b. The 4-node is black b with red children a and c. Red children have black children, so a black root absorbs at most two red children, giving at most three keys. Each contracted node contributes one black node to a path; equal root-to-NIL black counts imply equal external-leaf depths.





DS10-E10 • Comparison costs and rotation counts

Every successful or unsuccessful search compares the real key once, so the expectation is .4+.1+.5=1. Counting a dummy visit adds .1+.5=.6, giving 1.6. After insertion of 30, 10, 20, rotate left at 10, then right at 30. This is one LR double-rotation repair consisting of two primitive rotations.

References and attribution

Lin Cing, handwritten data structures notes. Sleator and Tarjan, Self-Adjusting Binary Search Trees, JACM 32(3), 1985, § 2: bottom-up splay and the access lemma. Numeric diagrams use this chapter's own keys and unit weights. The MIT 6.006 2020 index, L7, identifies AVL material. Princeton balanced search trees uses LLRB and its insertion/deletion code must not be mixed with general red-black conventions.

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study